HHE Kombat: Benchmarking Hybrid Homomorphic Encryption Schemes*

Hossein Abdinasibfar[1], Camille Nuoskala[1] and Antonis Michalas[1]

Tampere University, Tampere, Finland,
{hossein.abdinasibfar,camille.nuoskala,antonios.michalas}@tuni.fi

Abstract. Hybrid Homomorphic Encryption (HHE) is emerging as a practical alternative to fully homomorphic encryption by offloading computational overhead to the cloud. Despite the growing number of HHE schemes and implementations, no unified evaluation methodology currently exists. In this work, we present a comprehensive and reproducible benchmarking framework covering both standard and HE-friendly HHE schemes. We analyze and evaluate 19 open-source HHE frameworks, comprising 218 distinct ciphers' benchmarks, across diverse HE libraries and programming languages. To ensure fair comparison, we interpret the main results under an HHE-128 security target, separating standardized or author-claimed 128-bit settings from below-target measurements. Our contributions include a unified repository, a language-agnostic benchmarking tool, and detailed metrics on runtime and memory usage. The results offer actionable insights into the security-aware performance trade-offs of each design and lay the groundwork for standardizing future HHE evaluations.

Keywords: Applied Cryptography · HE-Friendly Ciphers · Homomorphic Encryption · Hybrid Homomorphic Encryption

1 Introduction

As computer security has grown even more sophisticated, so has the *art* of cryptography. It has been a while since Rivest *et al.* [RAD78] and other great cryptographers started thinking of ways to create advanced cryptographic schemes that enable computations on encrypted data. One of the first challenges they tried to address was how to create schemes that allow the computation of the sum of two numbers. Interestingly, addition happens to be one of the first mathematical operations humans practice. However, in cryptography, we still struggle to perform this fundamental operation within a system that, in many respects, remains a mystery to us. Currently, the most widely used techniques for this are Homomorphic Encryption (HE) [Gen09] and Functional Encryption (FE) [BSW11], with the former being more extensively studied.

Despite extensive research on HE, the practical adoption of such schemes in real-world applications is hindered by high computational, storage, and communication requirements. It is also worth noting that, in its early days, this novel cryptographic technique was nearly dismissed by the scientific community as impractical. However, with advancements in the field and increasing hardware capabilities, we can say that we are not entirely outside the realm of possibility.

In an HE scheme, a user begins by generating a public/private key pair $(\mathtt{pk}, \mathtt{sk})$ and an evaluation key $\mathtt{evk}$. Then, given two ciphertexts c_1, c_2 encrypting messages x_1 and x_2 respectively and the evaluation key $\mathtt{evk}$, it is possible to compute $f(c_1, c_2)$, where f is a

*This work was funded by the PiQASO EU research project (No. 101190366).

function associated either with addition or multiplication. Moreover, what is fascinating about HE is that in computing $f(c_1, c_2)$, there are no leaks about the underlying plaintexts x_1 and x_2 while decrypting the result is only feasible by possessing the secret key `sk`. Naturally, this opens up tremendous possibilities, as, for the first time, it becomes possible not only to outsource data, applications, and services but also *computations* to the cloud in a privacy-preserving manner. Despite its potential and the field's recent development, several challenges remain that prevent HE from being adopted by many services. Encrypting large amounts of data homomorphically requires significant resources, which prevents, at least, the use of HE on resource-constrained devices. To address part of the inefficiencies inherent in HE, researchers have recently introduced a novel transciphering technique known as Hybrid Homomorphic Encryption (HHE) [ANM26].

In an HHE scheme, a user encrypts data locally using a symmetric key `K` of a private-key encryption scheme `SKE`. Subsequently, `K` is encrypted under HE's public key `pk` and is outsourced to the cloud along with the ciphertexts and the evaluation key `evk`. Upon reception, the cloud can transform the symmetrically encrypted data into homomorphic ciphertexts and operate on them. This promising approach significantly reduces client-side computation costs by moving the most expensive computations to the cloud, where powerful machines handle data processing and storage.

HHE Potential

In theory, HHE offers an exciting direction to pursue. More precisely, the obvious advantage of HHE is that it can, in principle, allow *any* device to run HE. Hence, overcoming one of the main limitations of HE. Therefore, HHE can serve as a catalyst for identifying something that inspires the next step in building online services that provide real, privacy-respecting tools.

Comparison to Similar Works. HHE has garnered significant interest in recent years. By 2025, three studies had examined HHE schemes. Abdinasibfar *et al.* [ANM26] initiated this line of work by establishing the fundamental components, formal and security definitions, and attack taxonomy for HHE constructions, while also evaluating PASTA [DGH+23], HERA [CHK+21], and Rubato [HKL+22] within a single programming environment. Thakur *et al.* [TKLP25] later surveyed HHE schemes by their underlying HE frameworks and reported benchmarking results drawn from the original studies. Niu *et al.* [NWH+25] proposed a broader taxonomy covering both `SKE` and `HE` components, benchmarking 16 schemes using available open-source implementations, including standard and HE-friendly ciphers. Building on Abdinasibfar *et al.* [ANM26], they also included Masta [HKC+20] and AES-CKKS [ADE+23, NWH+25] alongside PASTA, HERA, and Rubato, and evaluated them on Chi-squared testing and convolutional neural network (CNN) inference.

However, their benchmarking lacks clarity on throughput calculations for HHE schemes. Since available frameworks use different languages, direct execution-time comparisons can yield inaccurate throughput estimates. Moreover, while runtime is important, many studies omit practical metrics such as memory usage and modular `SKE`/`HE` implementations, which are essential for enabling broader analyses across different ciphers.

1.1 Contributions

This paper maps the current landscape of HHE from a practical point of view. We bring together available open-source implementations, describe their differences, and provide a reproducible benchmarking setup so that researchers can compare schemes and evaluate new ones on the same footing. Our contribution can be summarized as follows:

C1. Collect and package a broad set of publicly available, up-to-date open-source implementations of state-of-the-art HHE schemes.

C2. Review, evaluate, and group the schemes and implementations using practical criteria, namely flexibility and applicability to real-world applications.

C3. Propose a single benchmarking method that works across programming languages and reports comparable metrics, including runtime, memory use, and throughput.

C4. Release an extensible benchmarking tool-chain that makes it easy to add and benchmark additional HHE schemes.

C5. Present and interpret benchmark results across schemes, and highlight insights, open problems, and promising research directions.

1.2 Organization

This paper is structured as follows: Section 2 introduces essential background on the core components of HE and HHE ciphers. It defines HE and HHE and outlines their general functionality, which is required for understanding the evaluation components. Section 3 presents the paper's core part: benchmarking HHE schemes. It outlines our methodology for collecting HHE implementations into an all-in-one repository and presents the current state of HHE development. It describes our benchmarking approach and tool, and reports the evaluation results. Finally, Section 4 discusses the findings and highlights key takeaways and directions for future research.

2 Background

In this section, we introduce our notation and provide a high-level overview of HE and HHE concepts, including their definitions and correctness. We then describe the challenges that prevent HE schemes from being practical in real-world applications and, finally, explain how the HHE flow addresses these challenges.

2.1 Notation

Vectors are in lowercase bold, and matrices are in uppercase. Let $[n]$ be $\{1, \ldots, n\}$, $[\cdot]_q$ denotes mod q reduction. We define λ and μ as the security parameters. We use $\mathbb{Z}_p$ for the group of integers mod p and $\mathbb{F}_p$ for the corresponding field. The plaintext and ciphertext spaces are $\mathcal{P}$ and $\mathcal{C}$, respectively. Finally, `pk`, `sk`, and `evk` denote public, secret, and evaluation keys.

2.2 Homomorphic Encryption

Since the groundbreaking work by Gentry [Gen09], numerous HE schemes have emerged to handle diverse data types. Notably, recommended by Homomorphic Encryption Standard [ACC+21], Brakerski-Gentry-Vaikuntanathan (BGV) scheme [BGV12] supports modular arithmetic over finite fields, enabling computations on vectors of integers in $\mathbb{Z}_q$ with $q \geq 2$. Similarly, Brakerski/Fan-Vercauteren (B/FV) scheme [Bra12] operates over finite fields and facilitates computations on integer vectors. Cheon-Kim-Kim-Song (CKKS) scheme [CKKS17] allows approximate computations on vectors of real and complex numbers. Additionally, the Torus FHE (TFHE) scheme [CGGI16, CGGI17, CGGI20] employs boolean circuits and decision diagrams for low-precision integers [CJP21]. We now provide a generic definition of HE to help the reader follow the rest of the paper.

Def. 2.1: General HE Definition

An $\texttt{HE} := (\texttt{KeyGen}, \texttt{Enc}, \texttt{Eval}, \texttt{Dec})$ scheme is a tuple of four algorithms defined as follows:

1. $\texttt{KeyGen}\left(1^{\lambda}\right)$ given a security parameter λ and outputs the public key $\texttt{pk}$, the secret key $\texttt{sk}$ and the evaluation key $\texttt{evk}$;
2. $\texttt{Enc}\left(\texttt{pk}, \mathbf{m}\right)$ given the public key $\texttt{pk}$ and a message $\mathbf{m}$ and outputs a ciphertext $\mathbf{c_m}$;
3. $\texttt{Eval}\left(\texttt{evk}, f, \left(c_{m_1}, \ldots, c_{m_n}\right)\right)$ given the evaluation key $\texttt{evk}$, a function f and an n-tuple of ciphertexts $\left(c_{m_1}, \ldots, c_{m_n}\right)$ and outputs a ciphertext $\mathbf{c}^f$;
4. $\texttt{Dec}\left(\texttt{sk}, \mathbf{c}^f\right)$ given the secret key $\texttt{sk}$ and a ciphertext $\mathbf{c}^f$ and outputs $\mathbf{m}^f$.

Correctness: An HE scheme is correct if:

$$\Pr\Big[\texttt{Dec}\big(\texttt{sk}, \mathbf{c}^f\big) \neq f(m_1, \ldots, m_n) \;\Big|\; \big[(\texttt{pk}, \texttt{sk}, \texttt{evk}) \leftarrow \texttt{KeyGen}(1^{\lambda})\big] \\ \wedge \; \big[\mathbf{c}^f \leftarrow \texttt{Eval}\big(\texttt{evk}, f, (c_{m_1}, \ldots, c_{m_n})\big)\big]\Big] = negl\big(\lambda\big).$$

2.3 Hybrid Homomorphic Encryption

Within the context of HE, high computational costs and significant ciphertext expansion are two major challenges that real-world applications face. To tackle these issues, researchers in [NLV11] proposed a hybrid approach, where the plaintext m is symmetrically encrypted using a randomly chosen key $\texttt{K}$ by a client. The resulting ciphertext c_m is much smaller than a homomorphic ciphertext, with an encryption ratio of $|c_m|/|m| \approx 1$. The client then sends c_m along with homomorphically encrypted $\texttt{K}$ to a remote location, such as a cloud service provider. Here, c_m is *transformed* into a homomorphic ciphertext c^{evl} by evaluating the SKE decryption circuit. Building on the HHE construction of Dobraunig *et al.* [DGH+23], we introduce a general definition of HHE that includes an additional *encapsulation* algorithm ($\texttt{Encap}$). This algorithm generates the symmetric key and encrypts it into a homomorphic ciphertext.

Def. 2.2: General HHE Definition

1. $\texttt{KeyGen}\left(1^{\lambda}\right)$ takes as input a security parameter λ and generates the homomorphic keys $(\texttt{pk}, \texttt{sk}, \texttt{evk}) \leftarrow \texttt{HE.KeyGen}\left(1^{\lambda}\right)$, outputs $(\texttt{pk}, \texttt{sk}, \texttt{evk})$;
2. $\texttt{Encap}\left(\texttt{pk}, 1^{\mu}\right)$ takes as input the public key $\texttt{pk}$, a security parameter μ and computes the symmetric key $\texttt{K} \leftarrow \texttt{SKE.Gen}\left(1^{\mu}\right)$ and its homomorphic encryption $c_{\texttt{K}} \leftarrow \texttt{HE.Enc}(\texttt{pk}, \texttt{K})$, outputs $(\texttt{K}, c_{\texttt{K}})$;
3. $\texttt{Enc}(\texttt{K}, \mathbf{m})$ takes as input, the symmetric key $\texttt{K}$ and a message $\mathbf{m}$ and computes the ciphertext $\mathbf{c_m} \leftarrow \texttt{SKE.Enc}(\texttt{K}, \mathbf{m})$, outputs $\mathbf{c_m}$;
4. $\texttt{Decomp}(\texttt{evk}, c_{\texttt{K}}, \mathbf{c_m})$ takes as input the evaluation key $\texttt{evk}$, the homomorphically encrypted symmetric key $c_{\texttt{K}}$ and a symmetric ciphertext $\mathbf{c_m}$, computes $\mathbf{c_m^{evl}} \leftarrow \texttt{HE.Eval}\Big(\texttt{evk}, \texttt{SKE.Dec}, (c_{\texttt{K}}, \mathbf{c_m})\Big)$, outputs $\mathbf{c_m^{evl}}$;
5. $\texttt{Eval}\left(\texttt{evk}, f, \left(c^{evl}_{m_1}, \ldots, c^{evl}_{m_n}\right)\right)$ takes as input the evaluation key $\texttt{evk}$, a function f defined in the ciphertext space of $\texttt{HE}$ and a n-tuple of homomorphic ciphertexts $c^{evl}_{m_i}$ and computes $\mathbf{c}^f \leftarrow \texttt{HE.Eval}\left(\texttt{evk}, f, \left(c^{evl}_{m_1}, \ldots, c^{evl}_{m_n}\right)\right)$, outputs $\mathbf{c}^f$;
6. $\texttt{Dec}\left(\texttt{sk}, \mathbf{c}^f\right)$ takes as input the homomorphic secret key $\texttt{sk}$ and a ciphertext $\mathbf{c}^f$ and computes $\mathbf{m}^f \leftarrow \texttt{HE.Dec}\left(\texttt{sk}, \mathbf{c}^f\right)$, if the message $\mathbf{m}^f$ is valid, outputs $\mathbf{m}^f$; otherwise outputs $\perp$.

Correctness: Additionally, an HHE scheme is correct if:

$$\Pr\Big[\mathtt{Dec}\big(\mathtt{sk},\mathbf{c}^f\big) \neq f(\mathbf{m}) \;\Big|\; \big[(\mathtt{pk},\mathtt{sk},\mathtt{evk}) \leftarrow \mathtt{KeyGen}(1^\lambda)\big] \wedge \big[(\mathtt{K},c_\mathtt{K}) \leftarrow \mathtt{Encap}(\mathtt{pk},1^\mu)\big] \wedge \big[\mathbf{c}_\mathbf{m}^{\mathbf{evl}} \leftarrow \mathtt{Decomp}(\mathtt{evk},c_\mathtt{K},\mathbf{c_m})\big] \wedge \big[\mathbf{c}^f \leftarrow \mathtt{Eval}(\mathtt{evk},f,\mathbf{c}_\mathbf{m}^{\mathbf{evl}})\big]\Big] = \mathsf{negl}\big(\lambda,\mu\big).$$

The correctness of HHE is achieved if both HE and SKE are correct.

HHE vs. HE. Privacy-Enhancing Technologies (PETs), such as HE, face practical obstacles in real-world settings where data is collected from a large number of resource-constrained devices. Many Internet of Things (IoT) applications generate large data volumes that are processed with Machine Learning (ML) and Big Data Analysis at scale [KNDP23]. In this setting, providers and users require services that are cost-effective, reliable, private, and secure. In a pure HE-based design, constrained devices must store the user's public key and encrypt their data under HE before sending it to the service provider. This can be prohibitive for low-end clients and increases communication and storage costs due to large HE ciphertexts. Symmetric-key encryption is more efficient, but it does not protect data from a curious or malicious provider when decryption is needed for further processing. HHE addresses these limitations by combining the efficiency of SKE with the server-side functionality of HE. Each device encrypts its data under a symmetric key K, while K is homomorphically encrypted and sent to the provider only *once*. The provider therefore stores only symmetrically encrypted data, reducing client-side computation, communication, and storage overhead. Although HHE increases provider-side computation, this *trade-off* is often acceptable due to client-side savings and improved scalability.

HHE Transciphering. The homomorphic evaluation of the SKE cipher's decryption circuit, known as *transciphering*, is the most resource-intensive component of HHE frameworks. Notably, transciphering is also described using terms such as *decompression* or *decomp* within the context of HHE schemes.

Classes of HHE Schemes. More than a decade has passed since the first HHE proposals, including Homomorphic AES [GHS12] and LowMC [ARS+15]. Since then, HHE research has mainly followed two directions: *Standard Transciphering*, which improves transciphering for AES, and *HE-friendly Transciphering*, which designs new ciphers with low-depth decryption circuits for more efficient HHE. A practical understanding of HHE schemes therefore requires studying the underlying SKE ciphers. While AES [DR05, JV02] is widely deployed, HE-friendly ciphers are not yet standardized, and their design choices and security limitations must be considered jointly. We omit a detailed discussion here to maintain focus on benchmarking, as recent surveys already cover the relevant technical details and known attacks for most HHE ciphers [ANM26, NWH+25].

3 HHE Benchmarking Framework

This section reports benchmarks for 218 HHE implementations[1], which constitute the core contribution of this paper. We first describe our methodology for identifying and collecting publicly available open-source HHE repositories, then summarize the current state of practical HHE development. Based on these observations, we introduce a unified benchmarking utility for accurate and fair performance measurements across implementations in different programming languages. Finally, we present the benchmark results and discuss their implications and directions for future research.

Benchmarking Environment. All experiments were conducted on an MNX Triton platform [Cen23] in an Ubuntu environment with an Intel® Xeon® Gold 6130 CPU @ 2.10GHz

[1] 84 SKE.Enc (client-side) and 134 HHE.Decomp (server-side)

and 128GB RAM. We measured resource consumption for both `SKE` and `HHE` functionalities, including runtime, CPU cycles, and memory usage.

Open Science & Reproducible Research. To support open science and reproducible research, our evaluation code is publicly available[2], enabling independent verification and extension to additional schemes.

3.1 Current State of HHE Implementations

HHE research has produced many schemes, but few follow open science principles or provide usable open-source code. In cryptography, open-source code is essential for analyzing schemes, identifying vulnerabilities, proposing fixes, and integrating them into new designs or platforms. However, most HHE repositories lack maintenance and adaptability. To build a unified repository of available, up-to-date, open-source HHE implementations, we systematically reviewed the literature and repositories, excluding outdated versions in favor of newer implementations. For example, we selected LowMC from the HybridHE framework [DGH+23] rather than the original repository [ARS+15]. For recent schemes without publicly available code – such as *Thunderbird* [WLW+24] and *Faster Evaluation of AES using TFHE* [SMK22] – we contacted the authors but received no responses.

As shown in Table 1, we collected 12 HE-friendly and 7 Standard HHE frameworks, along with 2 reference AES implementations.

Table 1: Hybrid Homomorphic Encryption Repositories

Framework	Ciphers	Libraries	Language
HE-friendly Transciphering			
HybridHE	Agrasta, Rasta, Dasta, FiLIP, Hera, Kreyvium, LowMC, Masta, Pasta, Pasta$_{v2}$	HElib [HS20], SEAL [SEA23], TFHE [CGGI19]	C/C++
Fasta	Fasta, Rasta	HElib [HS20], PALISADE* [Pro25], TFHE [CGGI19]	C/C++
Filip$_{ind}$	FiLIP	FINAL [COS22]	C/C++
Filip$_{hat}$	FiLIP	FINAL [COS22]	C/C++
Margrethe	Margrethe	MOSFHET [GBA24]	C/C++
YuX	YuX$_2$, YuX$_p$	HElib [HS20]	C/C++
HHELand	Pasta, Hera, Rubato	Lattigo [Ins]	Golang
Elisabeth	Elisabeth	Concrete [Zam22a]	Rust
Filip*	FiLIP	Concrete [Zam22a]	Rust
FRAST	FRAST	TFHE-rs [Zam22b]	Rust
Transistor	Transistor	TFHE-rs [Zam22b]	Rust
TriKre	Trivium, Kreyvium	TFHE-rs [Zam22b]	Rust
Standard (AES) Transciphering			
TinyAES*	AES	tinyaes	C/C++
Openssl*	AES	OpenSSL	C/C++
Fregata	AES	TFHEpp [Mat20]	C/C++
Full-LUT	AES	TFHE [CGGI19]	C/C++
FbT*	AES	OpenFHE [ABBB+22]	C/C++
FFT-CBS	AES	TFHE-rs [Zam22b]	Rust
Hippogryph	AES	TFHE-rs [Zam22b]	Rust
P-enc	AES	TFHE-rs [Zam22b]	Rust
BtR	AES	Lattigo [Ins]	Golang

* The PALISADE library has been deprecated in favor of OpenFHE, which introduces compatibility changes; as a result, we were unable to compile Rasta.
* This implementation was developed to compare FiLIP and Elisabeth. We were unable to build it because it depends on a deleted or private branch of the original Concrete library.
* Well-known AES libraries used as baselines for comparison.
* We encountered repeated linker errors when building this repository using the provided instructions. We traced the issue to the base LUT lookup code and the transciphering function. We include the FbT code in our repository for future resolution.

Our implementation-language analysis (Table 2) shows that HE-friendly frameworks rely

[2] https://github.com/hosseinabdinf/HHE-Kombat

on nine libraries written in C/C++ (50%), Rust (41.6%), and Golang (8.3%), whereas Standard HHE frameworks use five libraries written in C/C++ (42.8%), Rust (42.8%), and Golang (14.3%).

Since comparing the underlying HE libraries is outside our scope, we refer the reader to [GMT23] for a comprehensive analysis across multiple HE backends.

Table 2: Programming language distribution

Framework Type	Total	C/C++	Rust	Golang
HE-friendly	12 (63.16%)	50%	41.6%	8.3%
Standard	7 (36.84%)	42.8%	42.8%	14.3%

3.2 Security-level Normalization

Parameter selection directly affects security and performance. In HHE, this issue is amplified by combining two cryptographic layers with distinct assumptions and attack surfaces: the HE layer and the underlying SKE or transciphering primitive [ANM26]. For an implementation i, let $\lambda_{\mathsf{HE}}^{(i)}$ and $\mu_{\mathsf{SKE}}^{(i)}$ denote the HE and claimed SKE security levels, respectively. We define the effective HHE security level as:

$$\lambda_{\mathsf{eff}}^{(i)} = \min\left(\lambda_{\mathsf{HE}}^{(i)}, \mu_{\mathsf{SKE}}^{(i)}\right).$$

As shown in Table 3, we interpret the main comparisons under an **HHE-128** target [3]: a benchmark is 128-bit normalized only if $\lambda_{\mathsf{eff}}^{(i)} \geq 128$. AES benchmarks use standardized 128-bit security [DR05, JV02], whereas HE-friendly ciphers use the claims of the original papers for the evaluated variants. Below-target benchmarks, such as Trivium and Transistor (80-bit variant), are kept as references but should not be read as 128-bit normalized HHE results.

Exact equality of HE parameters is not possible across the evaluated implementations. BGV/BFV, CKKS, and TFHE backends use different plaintext domains, modulus chains or bootstrapping parameters, packing layouts, and circuit-depth requirements. *We therefore normalize by security target and measured functionality, not identical parameters.* This follows general practice in selecting HE parameters according to a target security level and scheme-specific constraints, while recognizing that efficient and secure parameter selection remains implementation-dependent [ACC+21, BCC+24, KMR24].

For BGV/BFV/CKKS baseline benchmarks, the relevant HE parameters are the ring dimension, plaintext modulus or CKKS scale, coefficient-modulus chain, levels, key-switching settings, and bootstrapping settings when applicable. For TFHE-style benchmarks, the relevant parameters are the LWE/RLWE dimensions, noise distributions, key-switching settings, and bootstrapping settings. Larger HE parameters or deeper circuits generally increase runtime and memory, while below-target symmetric security can make a benchmark appear faster; hence, below-128 benchmarks are excluded from the HHE-128 interpretation.

3.3 Benchmarking Approach

Recent HHE surveys [ANM26, TKLP25, NWH+25] lack a comprehensive collection of frameworks with fair, reproducible benchmarks needed to assess performance and support further research. The survey in [ANM26] highlights evaluation challenges due to implementations across different languages and platforms. Although [NWH+25] benchmarks 16

[3] Full security-level normalization is provided in: https://github.com/hosseinabdinf/HHE-Kombat/blob/main/security_level_normalization.md.

Table 3: Security classification of ciphers used in benchmark

Class	Ciphers
Standard 128-bit[a]	Full-LUT [TCBS23], Fregata [WWL+23], FFT-CBS [WHS+24], P-enc [BPR24], Hippogryph [BBB+25b], AES-CTR [ADE+23, NWH+25]
Claimed 128-bit[b]	LowMC [ARS+15], Kreyvium [CCF+15, BOS23], Rasta [DEG+18], Agrasta [DEG+18], Dasta [HL20], Fasta [CIR22], FiLIP [MCJS19, MPP23, CDPP22], Elisabeth [CHMS22], FRAST [CCH+24], Margrethe [AGHM24], Masta [HKC+20], Pasta [DGH+23], Pasta$_{v2}$ [GLR+24], Hera [CHK+21], Rubato [HKL+22], YuX [LLC+24], Transistor-128 [BBB+25a]
Below 128-bit[c]	Trivium [BOS23], Transistor-80 [BBB+25a]

[a] Included in HHE-128 comparisons when the HE layer also targets at least 128-bit security.
[b] Included as 128-bit research-cipher benchmarks when the HE layer also targets at least 128-bit security; the symmetric-security classification follows the original cipher papers cited in the corresponding benchmark tables rather than standardization.
[c] Reported only as reference measurements; not used to support 128-bit normalized conclusions.

schemes, it ignores these differences and lacks a clear methodology for throughput measurements. Runtime measurements alone do not reliably capture throughput, especially across diverse implementations. Memory usage and the modularity of `SKE` and `HE` components are also essential but often overlooked.

To address these gaps, we built a benchmarking tool that measures performance and memory usage accurately. It supports *C/C++, Go, Python, and Rust* to work with diverse HHE frameworks. We use *RDTSC* [Mic21] to count CPU cycles, enabling fair comparisons across languages by avoiding inconsistencies in built-in timers. Unlike millisecond measurements, *RDTSC* provides high precision with minimal OS interference and consistent results under varying loads[4].

All experiments were run on the same machine with a fixed CPU frequency and process pinning to a single core. For ciphers with native multi-threading, we report those results separately. We calculated the computation throughput (i.e., the number of bits that a scheme can process per second) of both `SKE` and `HHE` based on the measured CPU cycles, using:

$$T_{\text{bps}} = \frac{\#\text{bits}}{\#\text{cycles}} \times \text{CPU clock frequency}.$$

To improve readability, we converted computation throughput values from *bit/s* to *KB/s* for `SKE`, as the original values were large. Likewise, we traced and measured memory consumption[5] of both `SKE` and `HHE.Decomp` during the execution of HHE schemes, using:

$$T_{\text{KB/bit}} = \frac{\text{memory usage (KB)}}{\#\text{bits}}.$$

This metric reflects the amount of memory (in KB) required to process a single bit in the HHE scheme.

[4]According to the official Golang documentation [Go24], the garbage collector (GC) incurs a fixed cost per cycle plus a marginal cost proportional to the live heap size. Hence, RDTSC in Go captures both computational and GC cycles, meaning our results reflect end-to-end execution rather than pure algorithmic cost. Interestingly, despite GC overheads, Golang-based implementations outperformed most C/C++ based schemes.

[5]Our benchmarking tool offers memory usage monitoring using three indicators: (i) the resident set size (RSS) from /proc/self/statm, which represents the process's currently occupied physical memory; (ii) the heap allocation (Alloc), representing live heap memory; and (iii) the peak physical memory usage (MaxRSS). Memory usage refers to either the instantaneous or peak live memory. This metric thus reflects the memory footprint per processed bit, not the total amount of memory allocated over time.

3.4 Benchmarking Results

As shown in 2.2, an HHE scheme comprises two main components: the client encrypts data using `SKE.Enc()`, and the server applies `HHE.Decomp()` to transcipher the ciphertext into an HE ciphertext. In our all-in-one repository, we integrated our benchmarking tool into frameworks exposing these components. For frameworks without a separate design for `SKE.Enc()` and `HHE.Decomp()`, we implemented the corresponding benchmarking logic.

To enable meaningful comparisons between HE and HHE, we also benchmark the underlying HE schemes. We report `HE.Enc()` results in Table 4 and `SKE.Enc()` results in Tables 5 to 8. We follow the same approach for AES by benchmarking standard AES encryption in Table 9, which enables comparison with HE-friendly ciphers. We then report `HHE.Decomp()` results for AES transciphering in Table 10, and the corresponding results for HE-friendly ciphers in Tables 11 to 14.

Algorithmic and implementation-level scope. The reported HHE.Decomp() results should be interpreted as measurements of concrete implementations rather than pure cipher-level complexity. A transciphering runtime depends jointly on the SKE's algebraic structure, the decryption circuit, the HE backend, the chosen parameters, and engineering factors such as library, language, memory layout, and threading. We control the measurement environment using the same hardware, fixed CPU frequency, process pinning, RDTSC-based cycle counts, and separate reporting for multi-threaded variants. However, cross-library comparisons still conflate algorithmic and implementation effects. Consequently, our strongest algorithmic conclusions are drawn from comparisons within the same HE backend or controlled variants of the same family, whereas cross-HE-backend rankings are deployment-oriented comparisons of available implementations.

All results follow a consistent table format. Each table reports the scheme or cipher name, N (the number of elements in the plaintext/ciphertext), $\log_2 P$ (the bit-length of each element), **Time** (runtime in milliseconds), **CPU** (CPU cycles), **TRP** (computational throughput), **Memory** (memory consumption), and **TRP** for memory usage. Additionally, for `HE` parameters, $\log_2 Q$ (total coefficient-modulus) and D (LWE/RLWE dimension). The N column in the performance tables denotes the processed elements or slots for the benchmark workload and should not always be read as the cryptographic dimension.

Benchmarking `SKE.Enc()`. We first report benchmarking results for pure HE encryption, which serves as a client-side baseline for comparison with `SKE.Enc()` in HHE schemes. As shown in Table 4, we evaluated four widely used HE schemes: *BGV*, *BFV*, *CKKS*, and *TFHE*. For BGV, BFV, and CKKS, we encoded data in full-coefficient mode before encryption. The results show that Lattigo's BGV implementation achieves the highest computational throughput at 11617 KB/s. However, HElib's BGV implementation reaches the highest memory throughput at 0.0036 KB/bit. In contrast, TFHE is substantially slower and less memory-efficient on the client side, which reinforces the motivation for HHE in resource-constrained settings.

Insight #1

These results confirm that direct HE encryption can already be used in some packed settings, but the choice of HE backend strongly affects whether this is viable for edge devices or large-scale data collection.

Table 5 presents benchmarking results for seven ciphers operating over the binary field $\mathbb{F}_2$ with TFHE as the underlying HE scheme: Agrasta (AG) [DEG+18], Dasta (DA) [HL20], FiLIP (FI) [MCJS19], Kreyvium (KR) [CCF+15], LowMC (LO) [ARS+15], and Rasta (RA) [DEG+18]. Among these, the 12-round Kreyvium instance achieves the highest computational throughput at 55 KB/s, which is approximately 11× faster than TFHE but

Table 4: Benchmark Results of `HE.Enc()` for HE schemes using different libraries

Scheme	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (KB/s)	**Memory** (KB)	**TRP** (KB/bit)
HElib [HS20] - Parameters: $D_{\text{RLWE}} = \{2^{14}, 2^{15}, 2^{15}\}, \log_2 Q = \{160 - 600\}$							
BGV [DGH+23]	32768	16	31	6.62×10^7	2028.7553	5428	0.0103
BGV [DGH+23]	65536	32	82	1.74×10^8	3096.7818	14340	0.0068
BGV [DGH+23]	65536	60	81	1.72×10^8	5872.2358	14340	**0.0036**
Seal [SEA23] - Parameters: $D_{\text{RLWE}} = \{2^{13}, 2^{14}, 2^{15}\}, \log_2 Q = \{218, 438, 881, 1740\}$							
BFV [DGH+23]	8192	16	6	1.46×10^7	2299.8894	1544	0.0117
BFV [DGH+23]	16384	32	22	4.68×10^7	2870.4624	4356	0.0083
BFV [DGH+23]	32768	60	81	1.71×10^8	2948.0913	15876	0.0080
TFHE [CGGI19] - Parameters: $D_{\text{LWE}} = 2^{9.2992}, D_{\text{RLWE}} = 2^{10}$							
TFHE [DGH+23]	5	16	2	4.19×10^6	4.8934	196	2.4500
Lattigo [Ins] - Parameters: $D_{\text{RLWE}} = \{2^{14}, 2^{15}\}, \log_2 Q = \{160 - 200\}$							
BGV [ANM26]	8192	64	5	1.16×10^7	**11617.4910**	19276	0.0367
BGV [ANM26]	16384	64	22	4.81×10^7	5588.5229	78384	0.0747
BGV [ANM26]	32768	64	95	2.00×10^8	2681.9839	343736	0.1639
CKKS [ANM26]	16384	64	22	4.66×10^7	5769.1587	75332	0.0718
CKKS [ANM26]	32768	64	75	1.58×10^8	3405.0483	304976	0.1454
CKKS [ANM26]	65536	64	354	7.43×10^8	1446.3059	1556300	0.3710

still slower than other HE schemes. For memory throughput, all Kreyvium instances and FiLIP (FI1216) perform best at 1.05 KB/bit—a 2× improvement over TFHE's 2.45 KB/bit.

Table 5: Benchmark Results of `SKE.Enc()` for $\mathbb{F}_2$ ciphers (HHEFramework [DW])

Cipher	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (KB/s)	**Memory** (KB)	**TRP** (KB/bit)
AG4 [DEG+18]	5	16	1.549	3.25×10^6	6.3141	112	1.40
AG4* [DEG+18]	5	16	1.988	4.17×10^6	4.9227	112	1.40
DA5 [HL20]	5	16	1.045	2.19×10^6	9.3671	120	1.50
DA5* [HL20]	5	16	9.114	1.91×10^7	1.0740	252	3.15
DA6 [HL20]	5	16	0.609	1.28×10^6	16.0693	100	1.25
DA6* [HL20]	5	16	4.398	9.21×10^6	2.2258	232	2.90
FI1216 [MCJS19]	5	16	34.564	7.24×10^7	0.2832	84	**1.05**
FI1280 [MCJS19]	5	16	9.296	1.95×10^7	1.0529	112	1.40
KR12 [CCF+15]	5	16	0.176	3.69×10^5	**55.5045**	84	**1.05**
KR12B [CCF+15]	5	16	0.177	3.72×10^5	55.0607	84	**1.05**
KR13B [CCF+15]	5	16	0.232	4.86×10^5	42.1840	84	**1.05**
LO14 [ARS+15]	5	16	108.479	2.27×10^8	0.0902	268	3.35
RA5 [DEG+18]	5	16	89.343	1.87×10^8	0.1096	356	4.45
RA5* [DEG+18]	5	16	90.232	1.89×10^8	0.1085	212	2.65
RA6 [DEG+18]	5	16	22.166	4.64×10^7	0.4416	264	3.30
RA6* [DEG+18]	5	16	26.042	5.46×10^7	0.3759	212	2.65

* This refers to the packed version of the cipher.

Table 6 shows results for HE-friendly ciphers over the prime field $\mathbb{F}_p$: Masta (MA) [HKC+20], Hera (HR) [CHK+21], Pasta (PA) [DGH+23], and Pasta$_{v2}$ (PA$_{v2}$) [GLR+24]. Among these, HR5 achieves the highest computational throughput at 16163 KB/s, 1.4× faster than the best HE scheme from Table 4. For memory throughput, PA3$_{v2}$ performs best at 0.0067 KB/bit, approximately 1.8× higher than pure HE.

Table 7 reports benchmarking results for ciphers operating over both $\mathbb{F}_2$ and $\mathbb{F}_p$: Elisabeth (EL) [CHMS22], Trivium [BOS23], FiLIP$_{ind}$ [MPP23], FiLIP$_{hat}$ [CDPP22], FRAST [CCH+24], Transistor [BBB+25a], Margrethe [AGHM24], Yux [LLC+24], and Fasta [CIR22]. Among the $\mathbb{F}_2$ ciphers, FRAST achieves the highest computational throughput at 2121 KB/s, 441× faster than TFHE encryption from Table 4. Trivium leads in

Table 6: Benchmark Results of `SKE.Enc()` for $\mathbb{F}_p$ ciphers (HHEFramework [DW])

Cipher	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (KB/s)	**Memory** (KB)	**TRP** (KB/bit)
MA4 [HKC+20]	200	16	2.733	5.73×10^6	143.2484	212	0.0663
MA4 [HKC+20]	200	32	3.199	6.70×10^6	244.7405	212	0.0331
MA4 [HKC+20]	200	60	4.474	9.37×10^6	328.1410	212	0.0177
MA5 [HKC+20]	200	16	0.962	2.02×10^6	406.8623	116	0.0363
MA5 [HKC+20]	200	32	1.053	2.21×10^6	743.2541	116	0.0181
MA5 [HKC+20]	200	60	1.397	2.93×10^6	1050.4781	116	0.0097
HR5 [CHK+21]	200	16	0.232	4.86×10^5	1687.0735	92	0.0288
HR5 [CHK+21]	200	32	0.088	1.85×10^5	8844.6257	92	0.0144
HR5 [CHK+21]	200	60	0.090	1.90×10^5	**16163.3260**	92	0.0077
PA3 [DGH+23]	200	16	5.470	1.15×10^7	71.5741	92	0.0288
PA3 [DGH+23]	200	32	7.718	1.62×10^7	101.4661	92	0.0144
PA3 [DGH+23]	200	60	11.898	2.49×10^7	123.4002	92	0.0077
PA4 [DGH+23]	200	16	0.648	1.36×10^6	603.6272	88	0.0275
PA4 [DGH+23]	200	32	0.676	1.42×10^6	1157.9302	88	0.0138
PA4 [DGH+23]	200	60	1.021	2.14×10^6	1437.5938	88	0.0073
$PA3_{v2}$ [GLR+24]	200	16	2.743	5.75×10^6	142.7267	80	0.0250
$PA3_{v2}$ [GLR+24]	200	32	3.819	8.00×10^6	205.0465	80	0.0125
$PA3_{v2}$ [GLR+24]	200	60	5.876	1.23×10^7	249.8589	80	**0.0067**
$PA4_{v2}$ [GLR+24]	200	16	0.567	1.19×10^6	691.3391	84	0.0263
$PA4_{v2}$ [GLR+24]	200	32	0.577	1.21×10^6	1355.7428	84	0.0131
$PA4_{v2}$ [GLR+24]	200	60	0.843	1.77×10^6	1740.9265	84	0.0070

memory throughput at 0.0044 KB/bit, a 556× improvement over TFHE.

Table 7: Benchmark Results of `SKE.Enc()` for other ciphers

Cipher	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (KB/s)	**Memory** (KB)	**TRP** (KB/bit)
$\mathbb{F}_2$ ciphers							
EL-2KS [CHMS22]	1	4	4.3193	9.05×10^6	0.1133	35.64	8.91
EL-1KS [CHMS22]	1	4	4.3193	9.05×10^6	0.1133	35.64	8.91
EL-2KS* [CHMS22]	1	4	3.5011	7.33×10^6	0.1398	35.48	8.87
EL-1KS* [CHMS22]	1	4	3.5011	7.33×10^6	0.1398	35.48	8.87
Trivium-B [BOS23]	1	64	29.6112	6.20×10^7	0.2645	0.28	**0.0044**
Trivium-S [BOS23]	1	64	29.6112	6.20×10^7	0.2645	0.28	**0.0044**
Kreyvium-B [BOS23]	1	64	0.2016	4.22×10^5	38.8517	0.32	0.0050
Kreyvium-S [BOS23]	1	64	0.2016	4.22×10^5	38.8517	0.32	0.0050
Filip_{ind} [MPP23]	1	4	0.0081	1.22×10^4	83.8903	52	13
Filip_{hat} [CDPP22]	10	2	2.4418	4.95×10^6	1.0367	116	5.8
FRAST [CCH+24]	64	4	0.0283	5.93×10^4	1107.5762	11.4	0.0445
FRAST* [CCH+24]	64	4	0.0148	3.09×10^4	**2121.0407**	11.4	0.0445
Transistor [BBB+25a]	4	4	0.9608	2.00×10^6	2.0457	4	0.25
Margrethe [AGHM24]	32	8	11.8721	2.49×10^7	2.6399	40	0.1562
YuX_{2^8}-1 [LLC+24]	192	8	0.3628	7.04×10^5	559.3564	20	0.0130
YuX_{2^8}-4 [LLC+24]	192	8	0.3839	7.37×10^5	534.1982	32	0.0208
YuX_{2^8}-16 [LLC+24]	320	8	0.5879	1.16×10^6	563.3308	32	0.0125
$\text{YuX}_{2^{16}}$-16 [LLC+24]	32	8	0.2599	4.83×10^5	135.8971	32	0.1250
$\mathbb{F}_p$ ciphers							
YuX_p-16 [LLC+24]	32	64	0.0574	6.87×10^4	**7642.5889**	40	**0.0195**
Fasta [CIR22]	3	1645	6.4391	1.34×10^7	94.0864	116	0.0235

* Denotes the multi-thread execution.

Among the $\mathbb{F}_p$ ciphers, YuX_p-16 reaches 7642 KB/s computational and 0.0195 KB/bit memory throughput, outperforming Fasta in both metrics. However, it remains 1.5× slower than BGV and 2× slower than HR5 computationally, and 5× and 7× less memory-efficient than BGV and RB2, respectively.

Table 8 reports Rubato (RB) [HKL+22] separately from other HE-friendly ciphers in Table 6. Benchmarks of Pasta, Hera, and Rubato with Lattigo show that RB2 achieves

10924 KB/s computational throughput, comparable to BGV and 1.4× slower than HR5. However, Rubato achieves superior memory throughput at 0.0026 KB/bit, making it 1.3× and 2.5× more efficient than pure HE and PA3$_{v2}$, respectively.

Table 8: Benchmark Results of `SKE.Enc()` for $\mathbb{F}_p$ ciphers (HHELand [ANM26])

Cipher	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (KB/s)	**Memory** (KB)	**TRP** (KB/bit)
PA3 [DGH+23]	128	16	97.3466	2.04×10^8	2.5747	60	0.0293
PA3 [DGH+23]	128	32	118.9825	2.49×10^8	4.2130	58	0.0141
PA3 [DGH+23]	128	60	140.0006	2.93×10^8	6.7129	59	0.0076
PA4 [DGH+23]	128	16	8.2427	1.72×10^7	30.4394	58	0.0283
PA4 [DGH+23]	128	32	9.0704	1.90×10^7	55.3196	56	0.0136
PA4 [DGH+23]	128	60	11.205	2.34×10^7	83.9839	58	0.0075
HR5 [CHK+21]	128	25	0.2006	3.99×10^5	2053.5895	20	0.0063
HR5 [CHK+21]	128	28	0.1274	2.41×10^5	3808.9059	20	0.0055
RB5 [HKL+22]	128	16	0.347	6.99×10^5	750.5382	53	0.0260
RB3 [HKL+22]	128	36	0.240	4.76×10^5	2482.5773	32	0.0070
RB2 [HKL+22]	128	64	0.106	1.92×10^5	**10924.1293**	22	**0.0026**

Finally, Table 9 reports pure AES encryption as a baseline for AES transciphering schemes. TinyAES [Kok24] achieves higher computational throughput (7100 KB/s) and lower memory usage (0.43 KB/bit) than OpenSSL [The03]. In comparison, HR5 is approximately 2× faster than TinyAES, while RB2 is 165× more memory efficient.

Table 9: Benchmark Results of `SKE.Enc()` for pure AES ciphers

Cipher	**Lang**	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (KB/s)	**Memory** (KB)	**TRP** (KB/bit)
OpenSSL [The03]	C/C++	16	8	0.0049	1.01×10^4	3257.1471	1272	9.9375
TinyAES [Kok24]	C/C++	16	8	0.0023	4.62×10^3	**7100.7358**	56	**0.4375**

Insight #2

Taken together, the client-side benchmarks show three clear trends: (1) TFHE encryption is a poor fit for constrained clients. (2) $\mathbb{F}_2$ ciphers improve substantially over TFHE but still do not match the best prime-field constructions. (2) $\mathbb{F}_p$ schemes such as Hera, Rubato-2, YuX$_p$-16, and Pasta$_{v2}$ provide the best overall trade-off between throughput and memory use.

Benchmarking `HHE.Decomp()`. Table 10 reports AES transciphering benchmarks. Among the six evaluated frameworks, Hippogryph [BBB+25b] achieves the highest computational throughput at 23 bit/s, approximately 16× slower than TFHE's 373 bit/s simple linear regression (SLR) baseline in Table 14. Fregata [WWL+23] performs best in memory throughput at 5.6 KB/bit, a 1.3× improvement over TFHE's 7.45 KB/bit. For CKKS-based AES-CTR transciphering [ADE+23, NWH+25], computational and memory throughput reach 180 bit/s and 46 KB/bit, respectively. In comparison, CKKS-based SLR evaluation achieves 174,567,691 bit/s and 0.135 KB/bit, making it several orders of magnitude more efficient in both metrics.

Note that the SLR results serve as a baseline to contrast transciphering with pure HE computation. AES decryption circuits are significantly more complex than simple linear regression, so the primary comparison remains between AES-based and HE-friendly transciphering schemes.

Noise and Correctness. Most HE schemes introduce noise during encryption to ensure security [ANM26]. This noise starts small but grows with homomorphic operations—additions sum noises while multiplications amplify it significantly. To maintain correct

Table 10: Benchmark Results of `HHE.Decomp()` for AES transciphering

Cipher	N	$\log_2 P$	Time (ms)	CPU (cycles)	TRP (bit/s)	Memory (KB)	TRP (KB/bit)
Full-LUT [TCBS23]	32	8	589835	1.24×10^{12}	0.4350	216740	846.6406
Full-LUT* [TCBS23]	32	8	48837	1.02×10^{11}	5.2544	224712	877.7813
Fregata [WWL+23]	16	8	187184	3.92×10^{11}	0.6854	728	**5.6875**
FFT-halfCBS [WHS+24]	16	8	40841	8.56×10^{10}	3.1416	19404	151.5938
FFT-tight [WHS+24]	16	8	43194	9.05×10^{10}	2.9704	8708	68.0313
P-enc [BPR24]	16	8	190965	4.00×10^{11}	0.6718	796.8	6.2250
Hippogryph [BBB+25b]	16	8	5520	1.16×10^{10}	**23.2433**	2196	17.1563
AES-CTR [ADE+23, NWH+25]	2048	128	1457943	3.05×10^{12}	**180.2361**	12285032	46.8636

* Denotes the multi-thread execution.

decryption, the noise must stay below a given bound (the **noise budget**). Bootstrapping achieves this by refreshing a ciphertext to reset its noise to an acceptable level. We refer readers to the work of Al Bawadi and Polyakov [ABP23] for details on bootstrapping.

For TFHE and CKKS schemes, the libraries include bootstrapping, and our transciphering benchmarks produced no decryption failures. For BGV and BFV schemes using HElib and SEAL, we report noise levels from the libraries' APIs in Table 11 and Table 12. In BGV/BFV-based HHE frameworks, the remaining noise budget indicates both decryption correctness and capacity for subsequent operations—smaller budgets constrain future computations more severely.

Table 11 reports benchmarking results for HE-friendly schemes over both binary ($\mathbb{F}_2$) and prime ($\mathbb{F}_p$) fields using HElib. Among the $\mathbb{F}_2$ ciphers, Fasta achieves the highest computational throughput at 21.9 bit/s, which is 13.6× slower than BGV-based SLR evaluation (297.8 bit/s) but slightly faster than Hippogryph. For memory throughput, YuX_{2^8}-1 performs best at 4.6 KB/bit, though 2.7× less efficient than BGV's 12.75 KB/bit; however, it suffers from noise overflow and therefore decryption failure.

Among the $\mathbb{F}_p$ schemes, PA4_{v2} reaches the highest computational throughput at 201 bit/s, closely matching BGV-based SLR evaluation. For memory throughput, YuX_p-16 performs best at 13.6 KB/bit, slightly better than BGV-based SLR.

Table 12 presents results for HE-friendly schemes over both binary ($\mathbb{F}_2$) and prime ($\mathbb{F}_p$) fields using SEAL. Among the $\mathbb{F}_2$ ciphers, AG4 achieves the highest computational throughput at 5.02 bit/s, which is 252× slower than BFV-based SLR (1265.39 bit/s). Among the $\mathbb{F}_p$ schemes, PA4_{v2} reaches the highest computational throughput at 660.38 bit/s, approximately half of BFV-based SLR evaluation. For memory throughput, PA4 performs best at 15.78 KB/bit, slightly better than BFV's 9.21 KB/bit.

Table 13 reports results for HE-friendly schemes using the Lattigo library. RB2 achieves the highest computational throughput at 4989.51 bit/s, which is approximately 69,288× slower than BGV-based SLR evaluation (345,658,132 bit/s). RB2 also performs best in memory throughput at 15.48 KB/bit, significantly better than both BGV-based (0.0089 KB/bit) and CKKS-based (0.0135 KB/bit).

Table 14 reports benchmarking results for HE-friendly schemes using the TFHE library. TFHE itself achieves a computational throughput of 373.14 bit/s. Among the evaluated ciphers, Margrethe reaches the highest computational throughput at 813.33 bit/s, approximately 2.18× faster than direct TFHE evaluation. For memory throughput, Trivium-B performs best at 0.99 KB/bit, significantly better than TFHE's 7.45 KB/bit.

Final Comparison. Table 15 summarizes the most efficient and practical HE-friendly ciphers for client-side `SKE.Enc()`. For binary field ($\mathbb{F}_2$) schemes, FRAST, YuX_{2^8}-16, FiLIP_{ind}, and Kreyvium-12 perform best. For prime field ($\mathbb{F}_p$), Hera-5, Rubato-2, YuX_p-16, Pasta4_{v_2}, and Masta-5 perform best. Note that the top $\mathbb{F}_p$ ciphers are several orders of magnitude more efficient than $\mathbb{F}_2$ ciphers on the client side. Rubato-2 also achieves the highest memory efficiency.

Table 16 summarizes the most efficient and practical HHE ciphers for server-side

Table 11: Benchmark Results of `HHE.Decomp()` using HElib

Cipher	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (bit/s)	**Memory** (KB)	**TRP** (KB/bit)	**Noise Budget**
BGV [HS20]	200	16	11000	2.30×10^{10}	291.5842	265236	82.8863	5
BGV [HS20]	200	32	28549	5.98×10^{10}	224.7037	573480	89.6063	25
BGV [HS20]	200	60	40383	8.46×10^{10}	**297.8574**	153040	12.7533	20
$\mathbb{F}_2$ ciphers								
AG4 [DEG+18]	5	16	49084	1.03×10^{11}	1.6337	200868	2510.8500	17
AG4* [DEG+18]	5	16	21568	4.52×10^{10}	3.7180	73864	923.3000	51
DA5 [HL20]	5	16	346065	7.25×10^{11}	0.2317	1063196	13289.9500	27
DA5* [HL20]	5	16	52964	1.11×10^{11}	1.5140	114684	1433.5500	5
DA6 [HL20]	5	16	251962	5.28×10^{11}	0.3183	711708	8896.3500	5
DA6* [HL20]	5	16	69570	1.46×10^{11}	1.1526	111608	1395.1000	4
FI1216 [MCJS19]	5	16	1847327	3.87×10^{12}	0.0434	222072	2775.9000	19
FI1280 [MCJS19]	5	16	1512555	3.17×10^{12}	0.0530	707208	8840.1000	2
KR12 [CCF+15]	5	16	620229	1.30×10^{12}	0.1293	739564	9244.5500	19
KR12B [CCF+15]	5	16	627057	1.31×10^{12}	0.1279	739568	9244.6000	19
KR13B [CCF+15]	5	16	729022	1.53×10^{12}	0.1100	934608	11682.6000	12
LO14 [ARS+15]	5	16	1341420	2.81×10^{12}	0.0598	2388508	29856.3500	13
RA5 [DEG+18]	5	16	351644	7.37×10^{11}	0.2280	1063076	13288.4500	17
RA5* [DEG+18]	5	16	52431	1.10×10^{11}	1.5294	125076	1563.4500	5
RA6 [DEG+18]	5	16	258341	5.41×10^{11}	0.3104	711548	8894.3500	5
RA6* [DEG+18]	5	16	71068	1.49×10^{11}	1.1283	120096	1501.2000	4
YuX_{2^8}-1 [LLC+24]	192	8	213434	4.47×10^{11}	7.2139	7119	**4.6348**	**Fail**
YuX_{2^8}-4 [LLC+24]	192	8	120619	2.53×10^{11}	12.7649	7841	5.1048	12
YuX_{2^8}-16 [LLC+24]	32	8	100669	2.11×10^{11}	2.5491	11997	46.8633	11
$\text{YuX}_{2^{16}}$-16 [LLC+24]	32	8	256291	5.37×10^{11}	1.0013	38084	148.7656	123
Fasta [CIR22]	5	359	82073	1.72×10^{11}	**21.9233**	107976	60.1538	241
Rasta [CIR22]	3	351	799269	1.67×10^{12}	1.3206	4006940	3805.2612	12
Rasta* [CIR22]	1	329	114354	2.40×10^{11}	2.8839	150068	456.1337	243
$\mathbb{F}_p$ ciphers								
MA4 [HKC+20]	200	16	51680	1.08×10^{11}	62.0653	413824	129.3200	18
MA4 [HKC+20]	200	32	148717	3.12×10^{11}	43.1367	985396	153.9681	45
MA4 [HKC+20]	200	60	239011	5.01×10^{11}	50.3257	698752	58.2293	14
MA5 [HKC+20]	200	16	44369	9.30×10^{10}	72.2923	234176	73.1800	11
MA5 [HKC+20]	200	32	133491	2.80×10^{11}	48.0566	524596	81.9681	14
MA5 [HKC+20]	200	60	193301	4.05×10^{11}	62.2262	629756	52.4797	50
HR5 [CHK+21]	200	16	28574	5.99×10^{10}	112.2521	189960	59.3625	2
HR5 [CHK+21]	200	32	43487	9.11×10^{10}	147.5190	233112	36.4238	32
HR5 [CHK+21]	200	60	61821	1.30×10^{11}	194.5679	338596	28.2163	25
PA3 [DGH+23]	200	16	35964	7.53×10^{10}	89.1877	402604	125.8138	31
PA3 [DGH+23]	200	32	40246	8.43×10^{10}	159.3980	167020	**26.0969**	2
PA3 [DGH+23]	200	60	145364	3.05×10^{11}	82.7467	969696	80.8080	20
PA4 [DGH+23]	200	16	17252	3.61×10^{10}	185.9235	146328	45.7275	15
PA4 [DGH+23]	200	32	52633	1.10×10^{11}	121.8836	341956	53.4306	25
PA4 [DGH+23]	200	60	72836	1.53×10^{11}	165.1419	435216	36.2680	17
PA3_{v2} [GLR+24]	200	16	33104	6.94×10^{10}	96.8917	691100	215.9688	32
PA3_{v2} [GLR+24]	200	32	35388	7.41×10^{10}	181.2790	172424	**26.9413**	7
PA3_{v2} [GLR+24]	200	60	126564	2.65×10^{11}	95.0381	1559272	129.9393	20
PA4_{v2} [GLR+24]	200	16	15952	3.34×10^{10}	**201.0648**	212628	66.4463	15
PA4_{v2} [GLR+24]	200	32	46464	9.73×10^{10}	138.0660	490884	76.7006	25
PA4_{v2} [GLR+24]	200	60	64316	1.35×10^{11}	187.0189	468316	39.0263	18
YuX_p-16 [LLC+24]	32	64	115799	2.43×10^{11}	17.7282	27894	**13.6201**	30

* This refers to the packed version of the cipher.

transciphering `HHE.Decomp()`. For standard transciphering, AES-CTR and Hippogryph perform best. Among HE-friendly ciphers over the binary field ($\mathbb{F}_2$), Filip_{hat}, Margrethe, and Fasta perform best. Over the prime field ($\mathbb{F}_p$), Rubato-2, PA4_{v2} (SEAL), and PA4_{v2} (HElib) perform best. Rubato-2 also achieves the highest memory efficiency across all server-side schemes. Latency also matters for server-side computations. Filip_{hat} transciphers input in 68.5 ms, making it suitable for *time-sensitive applications*. By contrast, Rubato-2 excels for applications that process *large data volumes*.

Looking across all transciperhing results, no single scheme dominates under every metric. AES transciphering is valuable for compatibility but remains costly. Binary-field HE-friendly designs can reduce latency and sometimes lower memory usage, but their throughput is usually modest. Prime-field HE-friendly designs provide the strongest

Table 12: Benchmark Results of `HHE.Decomp()` using SEAL

Scheme	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (bit/s)	**Memory** (KB)	**TRP** (KB/bit)	**Noise Budget**
BFV [SEA23]	200	16	2534	5.31×10^9	**1265.3900**	29472	9.2100	14
BFV [SEA23]	200	32	11462	2.40×10^{10}	559.6622	81188	12.6856	124
BFV [SEA23]	200	60	70743	1.48×10^{11}	170.0284	244780	20.3983	**393**
$\mathbb{F}_2$ ciphers								
AG4 [DEG+18]	5	16	15973	3.35×10^{10}	**5.0203**	250848	**3135.6000**	82
DA5 [HL20]	5	16	105993	2.22×10^{11}	0.7566	1010732	12634.1500	46
DA6 [HL20]	5	16	74831	1.57×10^{11}	1.0716	665648	8320.6000	32
FI1216 [MCJS19]	5	16	1282911	2.69×10^{12}	0.0625	667252	8340.6500	122
FI1280 [MCJS19]	5	16	1512555	3.17×10^{12}	0.0530	707208	8840.1000	109
KR12 [CCF+15]	5	16	462019	9.68×10^{11}	0.1736	975468	12193.3500	202
KR12B [CCF+15]	5	16	454943	9.53×10^{11}	0.1763	975468	12193.3500	**203**
KR13B [CCF+15]	5	16	488624	1.02×10^{12}	0.1641	1137264	14215.8000	195
LO14 [ARS+15]	5	16	509449	1.07×10^{12}	0.1574	1400292	17503.6500	67
RA5 [DEG+18]	5	16	106467	2.23×10^{11}	0.7532	1010724	12634.0500	46
RA6 [DEG+18]	5	16	75511	1.58×10^{11}	1.0620	665636	8320.4500	32
$\mathbb{F}_p$ ciphers								
MA4 [HKC+20]	200	16	13825	2.90×10^{10}	231.9983	71252	22.2663	32
MA4 [HKC+20]	200	32	64711	1.36×10^{11}	99.1349	222000	34.6875	223
MA4 [HKC+20]	200	60	328555	6.88×10^{11}	36.6101	843316	70.2763	**690**
MA5 [HKC+20]	200	16	46072	9.65×10^{10}	69.6198	156988	49.0588	386
MA5 [HKC+20]	200	32	45881	9.61×10^{10}	139.8189	156988	24.5294	98
MA5 [HKC+20]	200	60	246871	5.17×10^{11}	48.7235	613936	51.1613	482
HR5 [CHK+21]	200	16	20234	4.24×10^{10}	158.5234	142384	44.4950	332
HR5 [CHK+21]	200	32	20076	4.21×10^{10}	319.5394	142384	22.2475	29
HR5 [CHK+21]	200	60	115169	2.41×10^{11}	104.4414	542124	45.1770	383
PA3 [DGH+23]	200	16	10940	2.29×10^{10}	293.1893	78904	24.6575	96
PA3 [DGH+23]	200	32	50978	1.07×10^{11}	125.8416	234156	36.5869	339
PA3 [DGH+23]	200	60	66936	1.40×10^{11}	179.6984	234156	19.5130	41
PA4 [DGH+23]	200	16	4893	1.03×10^{10}	655.5198	50496	**15.7800**	25
PA4 [DGH+23]	200	32	24673	5.17×10^{10}	259.9992	172328	26.9263	214
PA4 [DGH+23]	200	60	137961	2.89×10^{11}	87.1866	656944	54.7453	**680**
PA3_{v2} [GLR+24]	200	16	10698	2.24×10^{10}	299.8150	89852	28.0788	96
PA3_{v2} [GLR+24]	200	32	50148	1.05×10^{11}	127.9231	245044	38.2881	339
PA3_{v2} [GLR+24]	200	60	66580	1.39×10^{11}	180.6591	245044	20.4203	41
PA4_{v2} [GLR+24]	200	16	4857	1.02×10^{10}	**660.3780**	54544	17.0450	25
PA4_{v2} [GLR+24]	200	32	24628	5.16×10^{10}	260.4721	164392	25.6863	215
PA4_{v2} [GLR+24]	200	60	138441	2.90×10^{11}	86.8846	656944	54.7453	**680**

Table 13: Benchmark Results of `HHE.Decomp()` using Lattigo [ANM26]

Cipher	N	$\log_2 P$	**Time** (ms)	**CPU** (cycles)	**TRP** (bit/s)	**Memory** (KB)	**TRP** (KB/bit)
BGV [Ins]	8192	64	1.5	3.19×10^6	**345658132**	4688	**0.0089**
BGV [Ins]	16384	64	5	1.05×10^7	209241696	15544	0.0148
BGV [Ins]	32768	64	20	4.23×10^7	104051935	82240	0.0392
PA3 [DGH+23]	128	16	11346	2.38×10^{10}	180.9458	958260	467.9003
PA3 [DGH+23]	128	32	23088	4.84×10^{10}	177.8386	1904672	465.0078
PA3 [DGH+23]	128	60	24081	5.04×10^{10}	319.6950	1911936	248.9500
PA4 [DGH+23]	128	16	5867	1.23×10^{10}	349.9128	486740	237.6660
PA4 [DGH+23]	128	32	11908	2.49×10^{10}	344.7971	793812	193.8017
PA4 [DGH+23]	128	60	25105	5.26×10^{10}	306.6575	1913312	249.1291
CKKS [Ins]	16384	64	6	1.26×10^7	**174567691**	14248	**0.0135**
CKKS [Ins]	32768	64	21	4.59×10^7	96023640	45744	0.0218
CKKS [Ins]	65536	64	82	1.72×10^8	51218642	165308	0.0394
HR5 [CHK+21]	32768	25	350539	7.34×10^{11}	2342.5920	13894156	16.9606
HR5 [CHK+21]	32768	28	374047	7.84×10^{11}	2458.8081	14791988	16.1219
RB5 [HKL+22]	32768	16	187503	3.93×10^{11}	2802.8822	14003684	26.7099
RB3 [HKL+22]	32768	36	291087	6.10×10^{11}	4062.3044	23073868	19.5599
RB2 [HKL+22]	32768	64	421323	8.83×10^{11}	**4989.5059**	32464256	**15.4801**

throughput, especially when paired with HE backends that exploit packing, but they often require larger working memory and are not always ideal for time-sensitive settings.

Table 14: Benchmark Results of `HHE.Decomp()` using TFHE

Cipher	N	$\log_2 P$	Time (ms)	CPU (cycles)	TRP (bit/s)	Memory (KB)	TRP (KB/bit)
TFHE [CGGI20]	5	16	214903	4.50×10^8	**373.1419**	596	7.45
AG4 [DEG+18]	5	16	529105	1.11×10^{12}	0.1515	1224	15.30
DA5 [HL20]	5	16	7544755	1.58×10^{13}	0.0106	4936	61.70
DA6 [HL20]	5	16	4387679	9.19×10^{12}	0.0182	1176	14.70
FI1216 [MCJS19]	5	16	1990386	4.17×10^{12}	0.0402	328	4.10
FI1280 [MCJS19]	5	16	2102727	4.41×10^{12}	0.0381	324	4.05
KR12 [CCF+15]	5	16	374268	7.84×10^{11}	0.2142	1308	16.35
KR12B [CCF+15]	5	16	372155	7.80×10^{11}	0.2154	1308	16.35
KR13B [CCF+15]	5	16	409541	8.58×10^{11}	0.1958	1512	18.90
LO14 [ARS+15]	5	16	7832808	1.64×10^{13}	0.0102	1380	17.25
RA5 [DEG+18]	5	16	7549863	1.58×10^{13}	0.0106	4868	60.85
RA6 [DEG+18]	5	16	4366494	9.15×10^{12}	0.0183	3176	39.70
Rasta [DEG+18]	1	351	5404274	1.13×10^{13}	0.0651	66332	188.98
Filip_{ind} [MPP23]	1	4	38.77	8.12×10^7	103.4233	12	3
Filip_{hat} [CDPP22]	10	2	68.58	1.44×10^8	292.5586	3432	171.6
EL-2KS [CHMS22]	1	4	4731	9.91×10^9	0.8474	248	62
EL-1KS [CHMS22]	1	4	6086	1.28×10^{10}	0.6587	240	60
EL-2KS* [CHMS22]	1	4	320	6.71×10^8	12.5148	261.28	65.32
EL-1KS* [CHMS22]	1	4	354	7.42×10^8	11.3199	276.56	69.14
Trivium-B [BOS23]	1	64	5302	1.11×10^{10}	12.0980	63.6	**0.9938**
Trivium-S [BOS23]	1	64	1562	3.27×10^9	41.0672	95.2	1.4875
Kreyvium-B [BOS23]	1	64	5562	1.17×10^{10}	11.5340	200.8	3.1375
Kreyvium-S [BOS23]	1	64	2026	4.25×10^9	31.6536	298.4	4.6625
FRAST [CCH+24]	64	4	5960	1.25×10^{10}	43.0557	812	3.1719
FRAST* [CCH+24]	64	4	5377	1.13×10^{10}	47.7244	1180	4.6094
Transistor80 [BBB+25a]	4	4	445	9.34×10^8	35.9655	712	44.50
Transistor128 [BBB+25a]	4	4	533	1.12×10^9	30.0849	932	58.25
Margrethe [AGHM24]	32	8	2087	4.37×10^9	122.9700	1439	5.6211
Margrethe* [AGHM24]	32	8	315	6.61×10^8	**813.3301**	34576	135.0625

* Denotes the multi-thread execution.

Table 15: Results of `SKE.Enc()` for Top Ciphers

Cipher	Lang	N	$\log_2 P$	Time (ms)	CPU (cycles)	TRP (KB/s)	Memory (KB)	TRP (KB/bit)
TinyAES [Kok24]	C/C++	16	8	0.002	4.62×10^3	7100.739	56	0.4375
$\mathbb{F}_2$ ciphers								
FRAST [CCH+24]	Rust	64	4	0.028	5.93×10^4	1107.576	11.4	0.0445
YuX_{2^8}-16 [LLC+24]	C/C++	320	8	0.588	1.16×10^6	563.331	32	0.0125
Filip_{ind} [MPP23]	C/C++	1	4	0.008	1.22×10^4	83.890	52	13
KR12 [CCF+15]	C/C++	5	16	0.176	3.69×10^5	55.505	84	1.05
$\mathbb{F}_p$ ciphers								
HR5 [CHK+21]	C/C++	200	60	0.090	1.90×10^5	16163.326	92	0.0077
RB2 [HKL+22]	Golang	128	64	0.106	1.92×10^5	10924.129	22	0.0026
YuX_p-16 [LLC+24]	C/C++	32	64	0.057	6.87×10^4	7642.589	40	0.0195
PA4_{v2} [GLR+24]	C/C++	200	60	0.843	1.77×10^6	1740.926	84	0.0070
MA5 [HKC+20]	C/C++	200	60	1.397	2.93×10^6	1050.478	116	0.0097

Insight #3

As a result, the choice of HHE scheme should not be framed as the search for a universal winner. Instead, it should be driven by the intended deployment scenario: latency-critical applications favor TFHE-style designs such as Filip_{hat} *or Margrethe, whereas throughput-oriented analytics favor packed prime-field constructions such as Rubato-2 or* PA4_{v2}.

Table 16: Results of `HHE.Decomp()` for Top Ciphers

Cipher	Lang	N	$\log_2 P$	Time (ms)	CPU (cycles)	TRP (KB/s)	Memory (KB)	TRP (KB/bit)
AES transciphering								
AES-CTR [NWH+25]	Golang	2048	128	1457943	3.05×10^{12}	180.2361	12285032	46.8636
Hippogryph [BBB+25b]	Rust	16	8	5520	1.16×10^{10}	23.2433	2196	17.1563
$\mathbb{F}_2$ transciphering								
Filip$_{hat}$ [CDPP22]	C/C++	10	2	68.58	1.44×10^{8}	292.5586	3432	171.6
Margrethe [AGHM24]	C/C++	32	8	2087	4.37×10^{9}	122.9700	1439	5.6211
Fasta [CIR22]	C/C++	5	359	82073	1.72×10^{11}	21.9233	107976	60.1538
$\mathbb{F}_p$ transciphering								
Rubato-2 [HKL+22]	Golang	32768	64	421323	8.83×10^{11}	4989.5059	32464256	15.4801
PA4$_{v2}$ [GLR+24] (SEAL)	C/C++	200	16	4857	1.02×10^{10}	660.3780	54544	17.0450
PA4$_{v2}$ [GLR+24] (HELib)	C/C++	200	16	15952	3.34×10^{10}	201.0648	212628	66.4463

4 Discussion

This study provides a reproducible, security-aware benchmark of 218 HHE cipher configurations from 19 open-source frameworks. The results confirm that HHE can reduce client-side encryption cost compared to direct HE, but they do not identify a universal best HHE scheme. Instead, they identify families of implementations preferable under different constraints. Performance conclusions are therefore qualified by security target, HE backend, algebraic domain, packing strategy, and implementation architecture.

Key insights of this work are the following:

(1) The current open-source HHE ecosystem is dominated by C/C++ and TFHE-based implementations, while BGV and CKKS are the main arithmetic backends represented in the evaluated frameworks.

(2) Under our HHE-128 interpretation, AES-128 and author-claimed 128-bit HE-friendly ciphers are compared only when the HE layer also targets at least 128-bit security; below-target variants are retained only as reference measurements. Since BGV/BFV, CKKS, and TFHE-style schemes use different plaintext domains, parameter sets, bootstrapping methods, and packing layouts, future work should study systematic HHE parameter selection rather than forcing identical low-level parameters.

(3) Benchmark differences reflect the combined effect of cipher and implementation architecture. Binary-field ciphers reduce Boolean non-linearity, but performance still depends heavily on bit-level evaluation, lookup-table evaluation, or programmable bootstrapping. Prime-field ciphers often align better with BGV/BFV/CKKS arithmetic and packing, improving throughput but potentially increasing memory. TFHE-style designs suit gate-oriented workloads and can offer lower latency for small inputs. CKKS-style packed designs achieve high throughput for batched workloads via SIMD packing of many values per ciphertext [SV14]. AES transciphering implementations are valuable for compatibility, but performance mostly reflects the chosen homomorphic AES evaluation strategy rather than a change in AES itself. Therefore, latency-oriented applications favor TFHE-style implementations such as Filip$_{hat}$, Margrethe, and Hippogryph; throughput-oriented analytics favor packed arithmetic such as Rubato and $Pasta_{v2}$; and compatibility-driven deployments may still prefer AES-based transciphering despite higher cost.

(4) Despite rapid algorithmic and architectural improvements in the HHE field, practical adoption remains limited; current real-world deployments are mostly restricted to basic tasks like simple machine learning inference [FNB+24], data aggregation [NKAM25], or χ^2 calculations [NWH+25]. Our all-in-one repository and application-oriented insights are designed to lower this barrier to entry, making the utilization of pioneering HHE schemes in complex, real-world applications a primary objective for future research.

References

[ABBB+22] Ahmad Al Badawi, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, et al. Openfhe: Open-source fully homomorphic encryption library. In *proceedings of the 10th workshop on encrypted computing & applied homomorphic cryptography*, pages 53–63, 2022.

[ABP23] Ahmad Al Badawi and Yuriy Polyakov. Demystifying bootstrapping in fully homomorphic encryption. *Cryptology ePrint Archive*, 2023.

[ACC+21] Martin Albrecht, Melissa Chase, Hao Chen, Jintai Ding, Shafi Goldwasser, Sergey Gorbunov, Shai Halevi, Jeffrey Hoffstein, Kim Laine, Kristin Lauter, et al. Homomorphic encryption standard. *Protecting privacy through homomorphic encryption*, pages 31–62, 2021.

[ADE+23] Ehud Aharoni, Nir Drucker, Gilad Ezov, Eyal Kushnir, and Hayim Shaul. E2e near-standard and practical authenticated transciphering. *Cryptology ePrint Archive*, 2023.

[AGHM24] Diego F Aranha, Antonio Guimaraes, Clément Hoffmann, and Pierrick Méaux. Secure and efficient transciphering for fhe-based mpc. *Cryptology ePrint Archive*, 2024.

[ANM26] Hossein Abdinasibfar, Camille Nuoskala, and Antonis Michalas. The hhe land: Exploring the landscape of hybrid homomorphic encryption. *ACM Computing Surveys*, 58(12):1–35, 2026.

[ARS+15] Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for mpc and fhe. In *Advances in Cryptology – EUROCRYPT 2015*, pages 430–454, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.

[BBB+25a] Jules Baudrin, Sonia Belaïd, Nicolas Bon, Christina Boura, Anne Canteaut, Gaëtan Leurent, Pascal Paillier, Léo Perrin, Matthieu Rivain, Yann Rotella, et al. Transistor: a tfhe-friendly stream cipher. *Cryptology ePrint Archive*, 2025.

[BBB+25b] Sonia Belaïd, Nicolas Bon, Aymen Boudguiga, Renaud Sirdey, Daphné Trama, and Nicolas Ye. Further improvements in aes execution over tfhe: Towards breaking the 1 sec barrier. *Cryptology ePrint Archive*, 2025.

[BCC+24] Jean-Philippe Bossuat, Rosario Cammarota, Ilaria Chillotti, Benjamin R Curtis, Wei Dai, Huijing Gong, Erin Hales, Duhyeong Kim, Bryan Kumara, Changmin Lee, et al. Security guidelines for implementing homomorphic encryption. *Cryptology ePrint Archive*, 2024.

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *Proceedings of the 3rd Innovations in Theoretical Computer Science Conference*, ITCS '12, page 309325, New York, NY, USA, 2012. Association for Computing Machinery.

[BOS23] Thibault Balenbois, Jean-Baptiste Orfila, and Nigel Smart. Trivial transciphering with trivium and tfhe. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 69–78, 2023.

[BPR24] Nicolas Bon, David Pointcheval, and Matthieu Rivain. Optimized homomorphic evaluation of boolean functions. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 302–341, 2024.

[Bra12] Zvika Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *Advances in Cryptology CRYPTO 2012*, pages 868–886, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[BSW11] Dan Boneh, Amit Sahai, and Brent Waters. Functional encryption: Definitions and challenges. In Yuval Ishai, editor, *Theory of Cryptography*, pages 253–273, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.

[CCF+15] Anne Canteaut, Sergiu Carpov, Caroline Fontaine, Tancrède Lepoint, María Naya-Plasencia, Pascal Paillier, and Renaud Sirdey. Stream ciphers: A practical solution for efficient homomorphic-ciphertext compression. Cryptology ePrint Archive, Paper 2015/113, 2015.

[CCH+24] Mingyu Cho, Woohyuk Chung, Jincheol Ha, Jooyoung Lee, Eun-Gyeol Oh, and Mincheol Son. Frast: Tfhe-friendly cipher based on random s-boxes. *IACR Transactions on Symmetric Cryptology*, 2024.

[CDPP22] Kelong Cong, Debajyoti Das, Jeongeun Park, and Hilder VL Pereira. Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 563–577, 2022.

[Cen23] Triton Data Center. Triton, 2023.

[CGGI16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachene. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In *international conference on the theory and application of cryptology and information security*, pages 3–33. Springer, 2016.

[CGGI17] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster packed homomorphic operations and efficient circuit bootstrapping for tfhe. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 377–408. Springer, 2017.

[CGGI19] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: Fast fully homomorphic encryption library. 2019.

[CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Tfhe: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[CHK+21] Jihoon Cho, Jincheol Ha, Seongkwang Kim, Byeonghak Lee, Joohee Lee, Jooyoung Lee, Dukjae Moon, and Hyojin Yoon. Transciphering framework for approximate homomorphic encryption. In Mehdi Tibouchi and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2021*, pages 640–669, Cham, 2021. Springer International Publishing.

[CHMS22] Orel Cosseron, Clément Hoffmann, Pierrick Méaux, and François-Xavier Standaert. Towards case-optimized hybrid homomorphic encryption. In *Advances in Cryptology – ASIACRYPT 2022*, pages 32–67, Cham, 2022. Springer Nature Switzerland.

[CIR22] Carlos Cid, John Petter Indrøy, and Håvard Raddum. Fasta–a stream cipher for fast fhe evaluation. In *Cryptographers Track at the RSA Conference*, pages 451–483. Springer, 2022.

[CJP21] Ilaria Chillotti, Marc Joye, and Pascal Paillier. Programmable bootstrapping enables efficient homomorphic inference of deep neural networks. In *Cyber Security Cryptography and Machine Learning*, pages 1–19, Cham, 2021. Springer International Publishing.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *International conference on the theory and application of cryptology and information security*, pages 409–437. Springer, 2017.

[COS22] KU Leuven COSIC. Final. Online: https://github.com/KULeuven-COSIC/FINAL, 2022.

[DEG+18] Christoph Dobraunig, Maria Eichlseder, Lorenzo Grassi, Virginie Lallemand, Gregor Leander, Eik List, Florian Mendel, and Christian Rechberger. Rasta: A cipher with low anddepth and few ands per bit. In *Advances in Cryptology CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 1923, 2018, Proceedings, Part I*, page 662692, Berlin, Heidelberg, 2018. Springer-Verlag.

[DGH+23] Christoph Dobraunig, Lorenzo Grassi, Lukas Helminger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Pasta: A case for hybrid homomorphic encryption. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 30–73, 2023.

[DR05] Joan Daemen and Vincent Rijmen. *Rijndael/AES*, pages 520–524. Springer US, Boston, MA, 2005.

[DW] Christoph Dobraunig and Roman Walch. Framework for hybrid homomorphic encryption. Online: https://github.com/IAIK/hybrid-HE-framework.

[FNB+24] Eugene Frimpong, Khoa Nguyen, Mindaugas Budzys, Tanveer Khan, and Antonis Michalas. Guardml: Efficient privacy-preserving machine learning services through hybrid homomorphic encryption. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, pages 953–962, 2024.

[GBA24] Antonio Guimarães, Edson Borin, and Diego F Aranha. Mosfhet: Optimized software for fhe over the torus. *Journal of Cryptographic Engineering*, 14(3):577–593, 2024.

[Gen09] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing*, STOC '09, page 169178, New York, NY, USA, 2009. Association for Computing Machinery.

[GHS12] Craig Gentry, Shai Halevi, and Nigel P. Smart. Homomorphic evaluation of the aes circuit. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012*, pages 850–867, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.

[GLR+24] Lorenzo Grassi, Fukang Liu, Christian Rechberger, Fabian Schmid, Roman Walch, and Qingju Wang. Minimize the randomness in rasta-like designs: How far can we go? application to pasta. In *International Conference on Selected Areas in Cryptography*, pages 207–238, 2024.

[GMT23] Charles Gouert, Dimitris Mouris, and Nektarios Georgios Tsoutsos. Sok: New insights into fully homomorphic encryption libraries via standardized benchmarks. *Proceedings on Privacy Enhancing Technologies*, 3:154–172, 2023.

[Go24] Go. A guide to the go garbage collector, 2024.

[HKC+20] Jincheol Ha, Seongkwang Kim, Wonseok Choi, Jooyoung Lee, Dukjae Moon, Hyojin Yoon, and Jihoon Cho. Masta: An HE-Friendly Cipher Using Modular Arithmetic. *IEEE Access*, 8:194741–194751, 2020.

[HKL+22] Jincheol Ha, Seongkwang Kim, ByeongHak Lee, Jooyoung Lee, and Mincheol Son. Rubato: Noisy ciphers for approximate homomorphic encryption. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 581–610. Springer, 2022.

[HL20] Phil Hebborn and Gregor Leander. Dasta–alternative linear layer for rasta. *IACR Transactions on Symmetric Cryptology*, pages 46–86, 2020.

[HS20] Shai Halevi and Victor Shoup. Design and implementation of helib: a homomorphic encryption library. *Cryptology ePrint Archive*, 2020.

[Ins] Tune Insight. Lattigo v6. Online: https://github.com/tuneinsight/lattigo.

[JV02] Daemen Joan and Rijmen Vincent. The design of rijndael: Aes-the advanced encryption standard. *Information Security and Cryptography*, 17:31–50, 2002.

[KMR24] Elena Kirshanova, Chiara Marcolla, and Sergi Rovira. Guidance for efficient selection of secure parameters for fully homomorphic encryption. In *International Conference on Cryptology in Africa*, pages 376–400. Springer, 2024.

[KNDP23] Tomas Kliestik, Elvira Nica, Pavol Durana, and Gheorghe H Popescu. Artificial intelligence-based predictive maintenance, time-sensitive networking, and big data-driven algorithmic decision-making in the economics of industrial internet of things. *Oeconomia Copernicana*, 14(4):1097–1138, 2023.

[Kok24] Kokke. Tiny aes-c. Online: https://github.com/kokke/tiny-AES-c, October 2024.

[LLC+24] Fen Liu, Yongqiang Li, Huiqin Chen, Lin Jiao, Ming Luo, and Mingsheng Wang. Yux: Finite field multiplication based block ciphers for efficient fhe evaluation. *IEEE Transactions on Information Theory*, 70(5):3729–3749, 2024.

[Mat20] Kotaro Matsuoka. TFHEpp: pure C++ implementation of TFHE cryptosystem. https://github.com/virtualsecureplatform/TFHEpp, 2020.

[MCJS19] Pierrick Méaux, Claude Carlet, Anthony Journault, and François-Xavier Standaert. Improved filter permutators for efficient fhe: Better instances and implementations. In *Progress in Cryptology INDOCRYPT 2019: 20th International Conference on Cryptology in India, Hyderabad, India, December 1518, 2019, Proceedings*, page 6891, Berlin, Heidelberg, 2019. Springer-Verlag.

[Mic21] Microsoft. Rdtsc instruction (x86/x64 intrinsics), 2021.

[MPP23] Pierrick Méaux, Jeongeun Park, and Hilder VL Pereira. Towards practical transciphering for fhe with setup independent of the plaintext space. *Cryptology ePrint Archive*, 2023.

[NKAM25] Khoa Nguyen, Tanveer Khan, Hossein Abdinasibfar, and Antonis Michalas. A privacy-centric approach: Scalable and secure federated learning enabled by hybrid homomorphic encryption. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 280–296. Springer, 2025.

[NLV11] Michael Naehrig, Kristin Lauter, and Vinod Vaikuntanathan. Can homomorphic encryption be practical? In *Proceedings of the 3rd ACM workshop on Cloud computing security workshop*, pages 113–124, 2011.

[NWH+25] Chao Niu, Benqiang Wei, Zhicong Huang, Zhaomin Yang, Cheng Hong, Meiqin Wang, and Tao Wei. Sok: Fhe-friendly symmetric ciphers and transciphering. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(3):583–613, 2025.

[Pro25] Palisade Project. Palisade: Homomorphic encryption library, 2025.

[RAD78] R L Rivest, L Adleman, and M L Dertouzos. On data banks and privacy homomorphisms, 1978.

[SEA23] Microsoft SEAL (release 4.1). https://github.com/Microsoft/SEAL, January 2023. Microsoft Research, Redmond, WA.

[SMK22] Roy Stracovsky, Rasoul Akhavan Mahdavi, and Florian Kerschbaum. Faster evaluation of aes using tfhe. *Poster Session, FHE. Org*, 2022.

[SV14] Nigel P Smart and Frederik Vercauteren. Fully homomorphic simd operations. *Designs, codes and cryptography*, 71(1):57–81, 2014.

[TCBS23] Daphné Trama, Pierre-Emmanuel Clet, Aymen Boudguiga, and Renaud Sirdey. A homomorphic aes evaluation in less than 30 seconds by means of tfhe. In *Proceedings of the 11th Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, pages 79–90, 2023.

[The03] The OpenSSL Project. Openssl: The open source toolkit for ssl/tls. www.openssl.org, April 2003.

[TKLP25] Indranil Thakur, Angshuman Karmakar, Chaoyun Li, and Bart Preneel. A survey on transciphering and symmetric ciphers for homomorphic encryption. Cryptology ePrint Archive, Paper 2025/093, 2025.

[WHS+24] Ruida Wang, Jincheol Ha, Xuan Shen, Xianhui Lu, Chunling Chen, Kunpeng Wang, and Jooyoung Lee. Fhew-like leveled homomorphic evaluation: Refined workflow and polished building blocks. *Cryptology ePrint Archive*, 2024.

[WLW+24] Benqiang Wei, Xianhui Lu, Ruida Wang, Kun Liu, Zhihao Li, and Kunpeng Wang. Thunderbird: Efficient homomorphic evaluation of symmetric ciphers in 3gpp by combining two modes of tfhe. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024(3):530–573, 2024.

[WWL+23] Benqiang Wei, Ruida Wang, Zhihao Li, Qinju Liu, and Xianhui Lu. Fregata: Faster homomorphic evaluation of aes via tfhe. In *International Conference on Information Security*, pages 392–412. Springer, 2023.

[Zam22a] Zama. Concrete: TFHE Compiler that converts Python programs into FHE equivalent. https://github.com/zama-ai/concrete, 2022.

[Zam22b] Zama. TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data, 2022. https://github.com/zama-ai/tfhe-rs.